

A Project Report on

"AI-powered Task Scheduler for Fog Computing"

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN THE PARTIAL FULFILMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE

OF

BACHELOR OF ENGINEERING (COMPUTER ENGINEERING)
(Academic Year: 2025-26)

SUBMITTED BY

Miss. Gunjan Shahade	**Exam Seat No:**
Mr. Om Sawant	**Exam Seat No:**
Mr. Atul Sanap	**Exam Seat No:**
Mr. Shailesh Pawar	**Exam Seat No:**

Under the guidance of

Prof. Madhuri Shinde

DEPARTMENT OF COMPUTER ENGINEERING

BHUJBAL KNOWLEDGE CITY Mumbai Educational Trust | Institute of Engineering

MET's Institute of Engineering,
Adgaon, Nashik-422003
SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE

BHUJBAL KNOWLEDGE CITY
Mumbai Educational Trust

Institute of Engineering

Certificate

This is to Certify that the project report entitles

"AI-powered Task Scheduler for Fog Computing"

Miss. Gunjan Shahade	**Exam Seat No:**
Mr. Om Sawant	**Exam Seat No:**
Mr. Atul Sanap	**Exam Seat No:**
Mr. Shailesh Pawar	**Exam Seat No:**

are bonafide students of this institute and the work has been carried out by them under the guidance of Prof. Madhuri Shinde and it is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University for the award of the degree of Bachelor of Engineering (Computer Engineering).

Project Guide	External
(Prof. Madhuri Shinde)	(Examiner)
H.O.D	Principal
(Dr. P. M. Yawalkar)	(Dr. V. P. Wani)

Date: / /

Acknowledgements

We have taken efforts in this project. However, it would not have been possible without the kind support and help of many individual and organizations. We would like to extend our sincere thanks to all of them. It gives us proud privilege to complete the project on **"AI-powered Task Scheduler for Fog Computing"**. We are highly indebted to our internal guide (in case of sponsored project) **Prof. Madhuri Shinde** for his guidance and constant supervision as well as for providing necessary information regarding the project and also for his support in completing the project.

We are also extremely grateful to our respected H.O.D. (Computer Department) **Dr. P. M. Yawalkar** and **Dr. Vaibhav Dabhade, Prof. Mahesh Korade** (Project Co-ordinator) for providing all facilities and every help for smooth progress of project work.

Miss.Gunjan Shahade
Mr. Om Sawant
Mr. Atul Sanap
Mr. Shailesh Pawar

Abstract

Fog computing is an emerging technology that brings cloud-like services closer to users, reducing delay and improving performance for Internet of Things (IoT) applications such as healthcare sensors, traffic cameras, and smart home systems, etc. However, the limited capacity of fog nodes makes it difficult to decide where and how tasks should be executed efficiently. Our project focuses on building an intelligent task scheduling strategy using Artificial Intelligence (AI) with Deep Reinforcement Learning (DRL). The system selects the best fog or cloud node for task execution by considering multiple factors such as node workload, distance, and task priority. Since these factors often conflict with each other, we apply a multi-objective optimization approach to balance them and achieve the best results. This propose idea of task scheduler helps reduce processing delay, transmission delay, and overall response time while improving resource utilization. An environment will be developed where IoT-generated tasks are dynamically scheduled, and system performance is evaluated against traditional methods. This leads to faster, more efficient, and reliable services for IoT applications compared to traditional scheduling methods.

Contents

1 Introduction **1**

2 Literature Survey **2**

2.1 Efficient Task Scheduling and Load Balancing in Fog Computing for Healthcare (Farheen et al., 2025) . . . 2

2.2 A Hybrid Task Scheduling Technique in Fog Computing Using Fuzzy Logic and DRL (Choppara and Mangalampalli, 2024) . . . 3

2.3 An Intelligent Scheduling Strategy in Fog Computing System (Ibrahim and Askar, 2023) . . . 3

2.4 A Comprehensive Review on Multi-Objective Optimization Techniques (Sharma and Kumar, 2022) . . . 4

2.5 Reinforcement Learning-Based Resource Management for Fog Computing (Tran-Dang et al., 2022) . . . 5

2.6 Deep Reinforcement Learning for Scheduling in Edge Computing-based Industrial IoT (Wu et al., 2021) . . . 5

2.7 Summary . . . 6

3 Problem Statment **7**

3.1 Objectives and Scope . . . 7

3.2 Summary . . . 8

4 Analysis **9**

4.1 Project Plan . . . 9

4.1.1 Project Plan for semester I . . . 9

4.2 Requirement Analysis . . . 11

4.2.1 Necessary Functions . . . 11

4.2.2 Desirable Functions 11
4.3 Summary 11

5 Design **12**
5.1 Software Requirement Specifications 12
5.1.1 Project Scope 12
5.1.2 Operating Environment 13
5.1.3 User Classes and Characteristics 14
5.1.4 Design and Implementation Constraints 14
5.2 System Architecture 15
5.2.1 IoT Device Layer 16
5.2.2 Fog Layer 17
5.2.3 Cloud Layer 17
5.2.4 AI Scheduling Layer 17
5.2.5 Performance Monitoring Layer 18
5.3 External Interface Requirement 19
5.3.1 User Interfaces 19
5.3.2 Hardware Interfaces 19
5.3.3 Communication Interfaces 19
5.4 Software System Attribute 20
5.5 Nonfunctional Requirement 20
5.6 Entity Relationship Diagram (ERD) 21
5.7 Summary 22

6 Modeling **23**
6.1 Use Case Diagram 23
6.2 Class Diagram 26
6.3 Sequence Diagram 27
6.4 Summary 28

7 Implementation and Results **29**
7.1 Module-wise Implementation 29
7.1.1 IoT Dataset Module 29
7.1.2 Task Environment Creation Module 30
7.1.3 DRL-Based Scheduling Module 30

7.1.4 Multi-Objective Optimization Module (NSGA-II) 30
7.1.5 Performance Monitoring Module 31
7.2 Implementation Snapshots 31
7.3 Overall Results and Analysis 33
7.3.1 Performance Improvements 33
7.3.2 Scalability Analysis 33
7.3.3 Optimization Results 33
7.4 Summary 34

8 Testing **35**
8.1 Testing Strategy 35
8.1.1 Unit Testing 35
8.1.2 Integration Testing 35
8.1.3 Performance Testing 36
8.1.4 Edge Case Testing 36
8.2 Test Cases 36
8.3 Summary 37

9 Technical Specifications **38**
9.1 Applications 38
9.1.1 Hardware Requirements 39
9.1.2 Software Requirements 39
9.2 Summary 40

10 Future Scope **41**

11 Conclusion **43**

Appendix **44**

A Achievements **44**

B Certificates **45**

C Plagiarism Report **48**

D Team Information **51**

Bibliography **52**

List of Figures

5.1 System Architecture Diagram 16
5.2 Entity Relationship Diagram of AI-powered Task Scheduler for Fog Computing 21

6.1 Use Case Diagram of AI-powered Task Scheduler for Fog Computing . . 25
6.2 Class Diagram of AI-powered Task Scheduler for Fog Computing 26
6.3 Sequence Diagram of AI-powered Task Scheduler for Fog Computing . . 27

7.1 AI-powered Task Scheduler for Fog Computing System Interface 31
7.2 Task Environment Creation 32
7.3 Performance Monitoring Dashboard 32

List of Tables

4.1 Project Plan for Semester I - AI-powered Task Scheduler for Fog Computing 10

8.1 Test Cases for System Validation 36

D.1 Team Information 51

Chapter 1

Introduction

In today's digital era, the rapid expansion of the Internet of Things (IoT) has revolutionized various domains such as healthcare, transportation, industrial automation, and smart homes. These IoT applications generate an enormous amount of heterogeneous and continuous data from sensors, devices, and edge nodes. Processing such massive data streams efficiently and in real time poses a significant challenge for traditional centralized cloud computing systems. While cloud computing offers vast computational power and virtually unlimited storage capacity, it often suffers from high latency, network congestion, and bandwidth limitations when handling time-sensitive IoT tasks. These issues make it difficult to meet the stringent requirements of applications that demand low-latency responses, reliability, and seamless connectivity

To address these challenges, fog computing has emerged as an intermediate paradigm that extends cloud capabilities closer to the network edge. By deploying computational resources, storage, and networking services near IoT devices, fog computing significantly reduces latency, enhances real-time data processing, and alleviates the bandwidth burden on centralized cloud servers. However, despite these advantages, fog nodes generally have limited computational and storage capacities compared to the cloud, making efficient task allocation and resource management a complex problem.To overcome this limitation, our proposed idea introduces an AI-powered Task Scheduler for Fog Computing that leverages Deep Reinforcement Learning (DRL) techniques..

Chapter 2

Literature Survey

The exponential growth of Internet of Things (IoT) devices has led to an increased need for efficient task scheduling and resource management in fog and cloud environments. Several research studies have explored Artificial Intelligence (AI)-based solutions, particularly Deep Reinforcement Learning (DRL), to enhance system performance by reducing latency, balancing load, and optimizing resource utilization. This section provides an overview of key existing works in this domain, highlighting their contributions, methodologies, limitations, and research gaps.

2.1 Efficient Task Scheduling and Load Balancing in Fog Computing for Healthcare (Farheen et al., 2025)

Farheen et al. proposed a Deep Reinforcement Learning (DRL)-based model focused on healthcare IoT systems. The framework optimized task scheduling and load balancing to minimize latency and energy consumption in fog-based healthcare applications. The system dynamically analyzed network parameters and allocated workloads to appropriate fog nodes, resulting in improved task throughput and reduced response times. The study demonstrated the effectiveness of DRL in real-world healthcare scenarios where reliability and quick response are critical.

However, the proposed system was domain-specific, designed solely for healthcare data processing. It lacked flexibility to handle general-purpose IoT tasks from other domains like traffic control or smart manufacturing. Additionally, the framework did not

incorporate multi-objective optimization techniques to balance multiple QoS factors. The work nonetheless validated the potential of DRL in task scheduling and motivated further research toward developing a generalized, multi-domain DRL framework applicable across heterogeneous fog computing systems. [1].

2.2 A Hybrid Task Scheduling Technique in Fog Computing Using Fuzzy Logic and DRL (Choppara and Mangalampalli, 2024)

This paper introduced a hybrid task scheduling model that integrates Takagi-Sugeno fuzzy logic with Deep Reinforcement Learning (DRL) to address the limitations of conventional schedulers. The fuzzy inference system was used to preprocess environmental parameters such as latency, energy, and resource utilization, which were then used by the DRL agent to make optimal scheduling decisions. The hybrid model, known as HTSFFDRL, achieved multi-objective optimization across several performance metrics including makespan, energy efficiency, cost, and fault tolerance.

While the hybrid model exhibited superior scheduling accuracy and adaptability, it introduced higher computational complexity and required significant processing resources. This made it less feasible for lightweight fog environments with constrained capabilities. Moreover, the model was evaluated in a controlled simulation rather than a practical deployment scenario. Despite these challenges, this work strongly supports the concept of combining AI with fuzzy or evolutionary optimization—a direction closely aligned with the objectives of the proposed AI-powered fog task scheduler.[2]

2.3 An Intelligent Scheduling Strategy in Fog Computing System (Ibrahim and Askar, 2023)

Ibrahim and Askar proposed a novel intelligent scheduling strategy combining Deep Reinforcement Learning (DRL) with the NSGA-II algorithm to enhance decision-making in fog-cloud architectures. The approach optimized multiple parameters such as load balancing, task distance, and priority, ensuring efficient allocation of IoT tasks between fog and cloud nodes. The integration of DRL allowed the system to learn

from dynamic network conditions, while NSGA-II handled the trade-offs among multiple objectives. The hybrid method achieved improved scheduling accuracy and lower task completion times compared to traditional algorithms

Although the proposed system demonstrated strong results in simulation, it was not tested in a real-time fog computing environment. Additionally, it lacked a user interface or monitoring dashboard, which limits its usability for live system management. The absence of additional objectives, such as energy optimization and fault tolerance, reduced its practical applicability in large-scale IoT deployments. Nevertheless, this study provided the conceptual foundation for hybrid AI-optimization frameworks that inspire the development of adaptive scheduling solutions like the one proposed in this project..[3]

2.4 A Comprehensive Review on Multi-Objective Optimization Techniques (Sharma and Kumar, 2022)

This paper presented an in-depth review of various Multi-Objective Optimization (MOO) techniques, such as NSGA-II, MOEA/D, and SPEA2, which are widely used in engineering and decision-making systems. The authors discussed the mathematical foundations of these algorithms, emphasizing their ability to handle trade-offs between conflicting objectives like latency, cost, and resource utilization. The study also compared the strengths and weaknesses of different MOO strategies in terms of convergence rate, diversity maintenance, and computational efficiency.

However, the review was largely theoretical and lacked practical integration with fog or IoT-based scheduling environments. It did not explore how MOO techniques can be combined with AI or reinforcement learning to dynamically adapt to real-time workloads. The absence of application-oriented models highlighted a research gap — the need to merge optimization algorithms with intelligent learning systems for more efficient fog-cloud coordination. This insight forms the basis for developing an AI-powered hybrid scheduling framework that leverages both DRL and multi-objective optimization.[4]

2.5 Reinforcement Learning-Based Resource Management for Fog Computing (Tran-Dang et al., 2022)

Tran-Dang et al. provided an extensive literature review of Reinforcement Learning (RL) approaches for managing computational resources in fog computing environments. The study systematically evaluated methods such as Q-learning, SARSA, and Deep Q-Networks (DQNs) for dynamic task offloading and load balancing. It highlighted how RL models can learn optimal policies through environment interaction without predefined scheduling rules, thereby improving flexibility and automation in fog networks. The paper also categorized RL techniques based on their scalability, complexity, and convergence behavior.

Despite offering a solid theoretical foundation, the paper was primarily conceptual and did not include experimental implementation or performance validation. It emphasized open research issues such as multi-agent collaboration, real-time learning, and multi-objective optimization, but lacked practical frameworks that address these challenges holistically. The survey motivated the need for hybrid AI models that combine reinforcement learning with optimization algorithms to achieve adaptive and efficient task scheduling in real-time fog environments.[5]

2.6 Deep Reinforcement Learning for Scheduling in Edge Computing-based Industrial IoT (Wu et al., 2021)

This study proposed a Deep Reinforcement Learning (DRL)-based approach for task scheduling in edge-enabled industrial IoT systems. The authors used a Deep Q-Network (DQN) to learn optimal scheduling decisions dynamically, aiming to minimize the overall execution delay of IoT tasks. The DRL agent continuously interacted with the environment, observed task states, and optimized the mapping of tasks to edge servers. The framework achieved significant improvements in reducing latency and improving scalability, demonstrating the potential of DRL for decentralized IoT environments

However, the model primarily focused on single-objective optimization, targeting only the reduction of execution delay. It did not consider multiple QoS (Quality of Service) parameters such as energy efficiency, task priority, or node load balancing. Moreover, the approach relied heavily on simulation-based experiments without deployment on real fog or cloud infrastructures. These limitations restrict its applicability to dynamic real-world IoT environments where multiple objectives must be optimized simultaneously.[**?**]

2.7 Summary

The reviewed literature reveals that while AI and DRL techniques have significantly advanced task scheduling and resource management in fog computing, most existing works focus on single-objective optimization or domain-specific implementations. Limited efforts have been made to develop a generalized, adaptive, and multi-objective scheduling framework that dynamically allocates IoT tasks between fog and cloud nodes based on real-time conditions. To overcome these limitations, the proposed system integrates Deep Reinforcement Learning (DRL) with multi-objective optimization to design an intelligent scheduler that minimizes latency, balances workloads, and ensures efficient resource utilization in diverse IoT environments.

Chapter 3

Problem Statment

IoT applications generate massive data, but sending all of it to the cloud causes high latency, bandwidth issues, and overload. Fog computing reduces delay by processing data closer to users, but limited resources make task scheduling a challenge. To solve this, we propose an AI-powered scheduler using Deep Reinforcement Learning and multi-objective optimization to allocate tasks efficiently between fog and cloud nodes.

3.1 Objectives and Scope

Objectives

- To understand the Fog Scheduling : Study how fog computing works as a bridge between IoT devices and the cloud.
- To analyze techniques like **Deep Reinforcement Learning (DQN), multi-objective optimization** for smart scheduling.
- To implement intelligent scheduling system : Develop a system that automatically assigns tasks to fog or cloud nodes.
- Evaluate results of proposed system : Test the system using performance metrics like task completion time, latency, CPU load, and communication delay.

Scope

The scope of this project focuses on developing an **AI-powered Task Scheduler for Fog Computing** that ensures efficient resource utilization and real-time task execution across IoT environments. The proposed system intelligently allocates computational tasks between fog and cloud nodes to minimize latency, balance workload distribution, and optimize overall system performance. It incorporates advanced **Deep Reinforcement Learning (DRL)** techniques to enable adaptive and data-driven decision-making for task scheduling in dynamic network conditions. The scheduler is designed to provide **intelligent task management** for fog nodes, ensuring timely execution of IoT tasks while maintaining optimal use of available resources. Additionally, the project integrates a **performance evaluation dashboard** that enables real-time monitoring of key metrics such as CPU utilization, latency, and task completion time, providing transparency and system reliability. Through this approach, the project aims to enhance scalability, reduce delays, and achieve seamless coordination between fog and cloud infrastructures for next-generation IoT applications.

3.2 Summary

This chapter described the problem of high latency and bandwidth usage in cloud-based IoT systems and proposed fog computing as a solution for real-time data processing. It outlined the development of an **AI-powered Task Scheduler** that uses **Deep Reinforcement Learning (DRL)** and **multi-objective optimization** for efficient task allocation. The objectives focused on understanding fog scheduling, implementing intelligent decision-making, and evaluating performance metrics. The project scope emphasized improving resource utilization, reducing delays, and ensuring seamless coordination between fog and cloud environments

Chapter 4

Analysis

This chapter provides an overview of the project analysis and development plan adopted for the implementation of the AI-powered Task Scheduler for Fog Computing. The project follows the Rapid Application Development (RAD) model combined with the Model-View-Controller (MVC) architectural pattern to enable faster iteration, modular system design, and continuous improvement through user feedback. The RAD approach ensures quick prototyping and early testing, while MVC provides a clean separation between data handling, decision-making logic, and visualization components.

In this system, the Model represents data from IoT devices, fog nodes, and cloud nodes; the Controller implements the AI-based scheduling logic using Deep Reinforcement Learning (DRL) and multi-objective optimization; and the View is responsible for the dashboard interface that visualizes task execution, CPU load, latency, and other performance metrics in real time.

4.1 Project Plan

4.1.1 Project Plan for semester I

The first semester emphasized the research and design phase, laying the foundation for the project. The focus areas included a deep study of fog and cloud computing, literature review of DRL and optimization algorithms, problem formulation, feasibility analysis, and creation of system design diagrams. This phase also covered identifying user and system requirements, defining the mathematical model, and preparing initial architecture and prototype concepts.

Table 4.1 lists the semester I activities with their corresponding duration and team responsibility

Table 4.1: Project Plan for Semester I - AI-powered Task Scheduler for Fog Computing

Phase	**Activity**	**Start Date**	**End Date**	**Group Members**
1	Selection of Project Topic and Research Area	15-07-2025	27-07-2025	Team
1	Problem Statement Finalization	23-08-2025	29-08-2025	Gunjan, Om
1	Definition of Project Objectives	02-09-2025	05-09-2025	Atul, Shailesh
1	Literature Survey on DRL and Fog Scheduling Techniques	06-09-2025	15-09-2025	Gunjan, Om
1	Functional Requirement Specification (FRS) Preparation	17-09-2025	22-09-2025	Team
1	Feasibility Study (Technical, Economic, Operational)	24-09-2025	30-09-2025	Shailesh, Atul
1	System Design and Architecture Planning	01-10-2025	08-10-2025	Om, Atul
1	Set Theory and Mathematical Model Formulation	10-10-2025	15-10-2025	Gunjan, Shailesh
1	UML Diagram Preparation (Use Case, Sequence, Activity)	16-10-2025	22-10-2025	Team
1	Prototype Design for DRL Environment Simulation	23-10-2025	30-10-2025	Gunjan, Om
1	Documentation and Project Stage I Report Submission	01-11-2025	11-11-2025	Team

4.2 Requirement Analysis

4.2.1 Necessary Functions

- IoT Task Input Module.
- AI-Based Task Scheduling.
- Resource Monitoring System.
- Multi-Objective Optimization Engine.
- Performance Dashboard.

4.2.2 Desirable Functions

- Dynamic Load Balancing.
- Energy Efficiency Module.
- Fault Tolerance and Recovery.
- Scalability Support.
- Alert and Logging System.

4.3 Summary

In this chapter, we discussed the requirement analysis and outlined the functional design necessary for the implementation of the AI-powered Task Scheduler for Fog Computing. The necessary functions describe the core operations essential for intelligent task scheduling, such as AI-driven decision-making, optimization, and performance monitoring. The desirable functions focus on enhancing system reliability, scalability, and energy efficiency to improve the overall user and administrator experience. These requirements provide a strong foundation for the design and development of the proposed system, ensuring it delivers efficient, adaptive, and real-time task scheduling across fog and cloud environments.

Chapter 5

Design

This chapter describes the Software Requirement Specification (SRS) for the AI-powered Task Scheduler for Fog Computing system. It explains the system architecture, external interface requirements, and the design constraints that guided the development. Additionally, it outlines the risk assessment strategy and includes data flow details that illustrate how data moves throughout the system components.

5.1 Software Requirement Specifications

The Software Requirement Specification (SRS) defines the scope of the project, operating environment, user characteristics, design limitations, and overall system architecture of the AI-powered Task Scheduler for Fog Computing system. It serves as a foundation for the software design and implementation process.

5.1.1 Project Scope

The AI-powered Task Scheduler for Fog Computing aims to address the challenges of high latency, bandwidth congestion, and cloud overload in traditional IoT–cloud environments. By introducing fog computing, computation is brought closer to IoT devices, allowing faster response and efficient real-time task execution.

The project focuses on developing an intelligent task scheduling system that leverages Deep Reinforcement Learning (DRL) and Multi-objective Optimization (NSGA-II) to allocate IoT-generated tasks dynamically between fog and cloud nodes. This ensures efficient resource utilization, reduced latency, and improved load balancing across

the network.

- Intelligently allocate IoT tasks between fog and cloud nodes to reduce latency and optimize resource usage.
- Implement an AI-driven scheduler using DRL for adaptive and data-driven task management.
- Integrate multi-objective optimization (NSGA-II) to balance parameters such as delay, load, and energy.
- Develop a performance dashboard for real-time monitoring of task execution and system performance.
- Enable scalability and fault tolerance for dynamic IoT environments.

5.1.2 Operating Environment

The system is designed to operate within a hybrid IoT–Fog–Cloud environment, ensuring efficient performance in both simulated and real-time network conditions.

Software Environment:

- Operating System: Windows 10 / 11 (64-bit)
- Programming Language: Python 3.9+
- Frameworks and Libraries: TensorFlow / PyTorch, OpenAI Gym, Pymoo / DEAP, Matplotlib / Plotly
- Database: MySQL / Firebase for task and performance storage
- Development IDEs: Jupyter Notebook, PyCharm, VS Code

Hardware Environment:

- Processor: Intel i5 / i7 or equivalent
- RAM: Minimum 8 GB (16 GB recommended)
- Storage: 512 GB SSD

- GPU: Integrated / Dedicated Graphics for DRL training
- Network: Stable Internet connection for IoT–Fog–Cloud communication

5.1.3 User Classes and Characteristics

The AI-powered Task Scheduler for Fog Computing system is designed for various categories of users and stakeholders:

- System Administrators: Monitor fog node performance, manage scheduling policies, and observe real-time metrics.
- IoT Application Developers: Integrate the scheduling framework into IoT applications for enhanced performance.
- Data Analysts and Researchers: Analyze system data and performance metrics for optimization studies.
- Enterprise Users: Utilize the scheduler for large-scale IoT environments such as healthcare, smart homes, and traffic systems.

5.1.4 Design and Implementation Constraints

Developing the AI-powered Task Scheduler for Fog Computing involves specific design and implementation constraints related to computational limits, real-time responsiveness, and system integration.

- Limited Resources: Fog nodes have restricted processing and storage capabilities compared to cloud servers.
- Dynamic Environment: Network load and IoT data rates change frequently, requiring adaptive decision-making.
- Training Complexity: DRL models require high computational resources and long training time.
- Latency Sensitivity: Real-time IoT applications demand rapid task assignment within milliseconds.

- Interoperability: Ensuring smooth communication between IoT devices, fog nodes, and cloud environments.

The following are the merits of the design implementation: -

- Low Latency: Tasks are processed closer to IoT devices, minimizing delay.
- High Resource Utilization: Efficient scheduling balances workload among fog and cloud nodes.
- Scalability: Easily adaptable for multiple fog nodes and large IoT networks.
- Reliability: Fault-tolerant scheduling ensures consistent system performance.
- Transparency: A real-time monitoring dashboard improves visibility of system operations.

5.2 System Architecture

In modern IoT environments, massive data generated by connected devices requires real-time analysis and intelligent decision-making. The architecture of the AI-powered Task Scheduler for Fog Computing is designed to ensure efficient task allocation, resource management, and latency reduction through an AI-driven, multi-layered framework.

The proposed architecture, as shown in Figure 5.1, is divided into five major layers IoT Device Layer, Fog Layer, Cloud Layer, AI Scheduling Layer, and Performance Monitoring Layer. Each layer performs a distinct role and collaborates seamlessly to achieve intelligent task scheduling, adaptive learning, and efficient resource utilization.

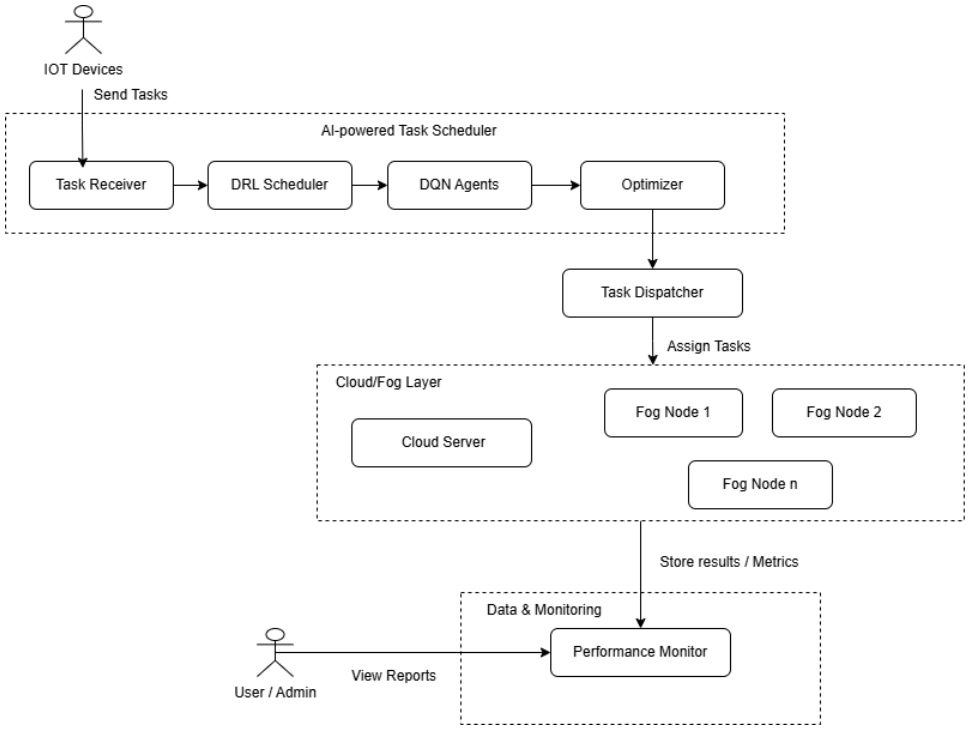


Figure 5.1: System Architecture Diagram

The proposed system architecture of the AI-powered Task Scheduler for Fog Computing ensures dynamic and efficient allocation of IoT-generated tasks. Each layer is designed to handle a specific set of functions—ranging from task generation and scheduling to optimization, processing, and performance monitoring.

5.2.1 IoT Device Layer

This layer is responsible for task generation and initial data transmission. Various IoT devices such as sensors, cameras, or smart appliances generate computational tasks based on real-time inputs. These tasks include parameters such as task size, priority, and required execution time before being sent to the fog nodes.

Components:

- IoT Sensors and Smart Devices
- Task Generator Module
- Data Transmission Interface

5.2.2 Fog Layer

The fog layer acts as the intermediate processing unit between IoT devices and cloud servers. It performs computation close to the data source to reduce latency and bandwidth usage. The layer uses local fog nodes to process high-priority tasks while forwarding complex ones to the cloud.

Functions:

- Receive and queue tasks from IoT devices
- Perform local computation for latency-sensitive tasks
- Maintain node resource status (CPU, memory, and load)
- Forward large or time-insensitive tasks to the cloud layer

5.2.3 Cloud Layer

The cloud layer provides centralized data storage, large-scale computation, and historical analysis. Tasks that cannot be efficiently handled by fog nodes are transferred to the cloud for execution.

Functions:

- High-capacity data storage and backup
- Processing of complex and non-real-time tasks
- Long-term data analytics and optimization model training
- System-level updates and synchronization with fog nodes

5.2.4 AI Scheduling Layer

This is the core decision-making layer of the system. It integrates Deep Reinforcement Learning (DRL) and Multi-objective Optimization (NSGA-II) to determine the optimal placement of tasks between fog and cloud nodes. The DRL agent learns continuously from system states such as resource availability, queue length, and task delay to make adaptive scheduling decisions.

Functions:

- Task Scheduling using Deep Reinforcement Learning (DQN/DRL)
- Multi-objective Optimization for balancing latency, cost, and load
- Adaptive Decision-Making under dynamic network conditions
- Reward and Policy Updating for continuous learning

5.2.5 Performance Monitoring Layer

This layer collects performance metrics, visualizes system behavior, and provides insights for optimization. It includes a web-based dashboard that tracks execution statistics such as CPU utilization, latency, throughput, and energy consumption. The layer also helps in validating scheduler performance and maintaining transparency.

Functions:

- Real-time Performance Monitoring and Visualization Dashboard
- Data Logging and Reporting
- Alert System for Node Overload or Task Failure
- Feedback Mechanism for DRL Model Retraining

In summary, the architecture of the AI-powered Task Scheduler for Fog Computing integrates IoT, fog, and cloud computing environments under an AI-driven scheduling system. Each layer contributes to reducing latency, balancing load, and improving reliability, ensuring real-time and efficient execution of IoT tasks across distributed nodes.

5.3 External Interface Requirement

5.3.1 User Interfaces

- **Web Dashboard:** Provides a real-time visualization interface to monitor task scheduling, CPU utilization, latency, and resource allocation across fog and cloud nodes.
- **Administrator Console:** Allows system administrators to configure fog nodes, manage scheduling policies, and monitor performance logs.
- **Simulation Interface:** Enables developers and researchers to simulate IoT task generation and evaluate the AI scheduler under various network conditions.

5.3.2 Hardware Interfaces

- **Fog Nodes:** Edge devices or microservers with limited computational resources responsible for handling latency-sensitive IoT tasks.
- **Cloud Servers:** High-performance nodes responsible for executing complex and computation-heavy tasks offloaded from fog nodes.
- **IoT Devices:** Sensors or smart devices generating real-time data tasks that are processed through the scheduler.

5.3.3 Communication Interfaces

The system communicates through secure RESTful APIs and TCP/IP protocols to ensure efficient interaction between IoT devices, fog nodes, and cloud servers. Data is transmitted via the internet using HTTPS, and communication latency is minimized through local fog processing.

5.4 Software System Attribute

- **Reliability:** The system ensures stable and accurate AI-based task scheduling using Deep Reinforcement Learning and multi-objective optimization.
- **Availability:** Fog and cloud coordination ensures high availability of services even during node failures or network interruptions.
- **Maintainability:** Each system component (AI agent, optimizer, and dashboard) can be updated independently without disrupting the overall architecture.
- **Scalability:** The scheduler can easily integrate additional fog nodes or IoT devices to accommodate larger workloads.

5.5 Nonfunctional Requirement

The **AI-powered Task Scheduler for Fog Computing** represents a complex AI-driven system that requires efficiency, adaptability, and reliability. The key non-functional requirements are as follows:

- **Performance:** The system must deliver low latency and high throughput by intelligently scheduling IoT tasks between fog and cloud nodes using DRL.
- **Usability:** The dashboard and simulation interfaces must be user-friendly, providing clear visualizations and performance metrics for monitoring and control.
- **Scalability:** The architecture should allow easy integration of new nodes and IoT devices without major reconfiguration or performance degradation.
- **Adaptability:** The DRL-based scheduler must dynamically respond to changes in workload, network congestion, and resource availability.
- **Security and Data Privacy:** All communication between fog and cloud nodes must be encrypted, and system logs must be securely stored to prevent unauthorized access.

5.6 Entity Relationship Diagram (ERD)

The Entity Relationship Diagram (ERD) of the **AI-powered Task Scheduler for Fog Computing** system is shown in Figure **??**. It illustrates the relationships between major entities such as IoT Devices, Tasks, Fog Nodes, Cloud Nodes, AI Agent, Optimizer and Performance Metrics. Each entity stores key attributes that enable efficient task scheduling, monitoring, and optimization across the fog–cloud infrastructure.

The ERD clearly demonstrates how tasks are generated by IoT devices, assigned by the AI scheduler to either fog or cloud nodes, and evaluated based on performance metrics such as latency, execution time, and CPU utilization. It defines both primary keys (PK) and foreign keys (FK) that maintain database consistency and ensure accurate task mapping.

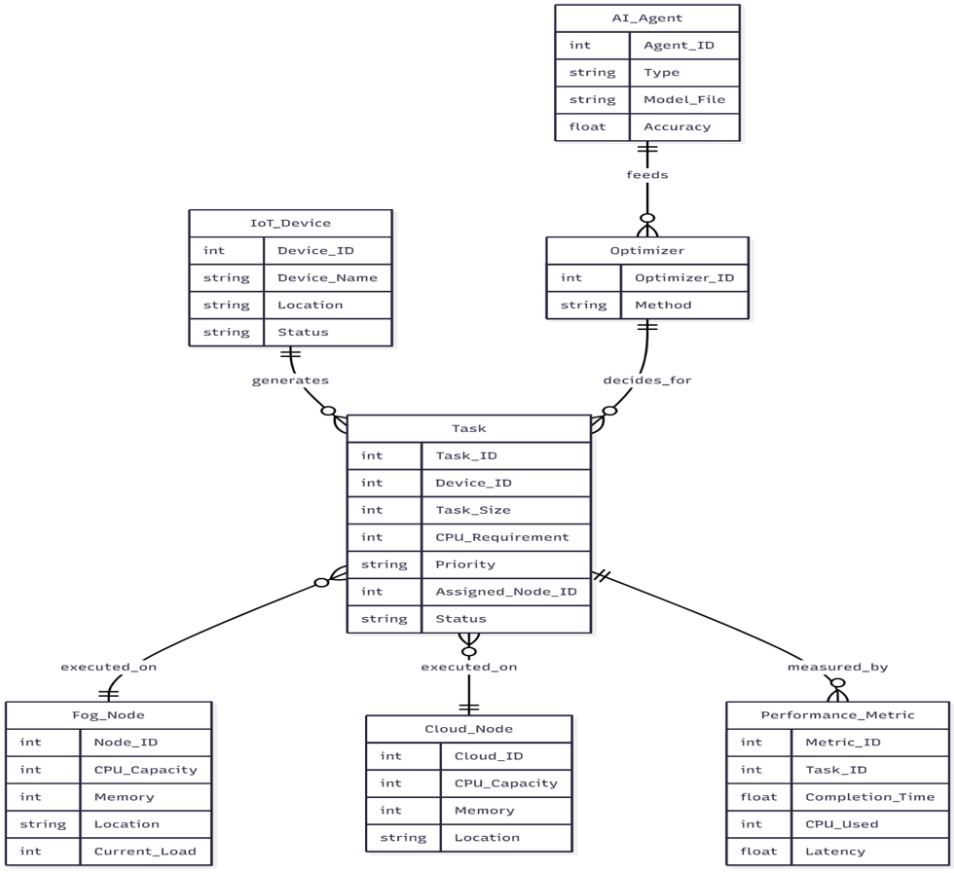


Figure 5.2: Entity Relationship Diagram of AI-powered Task Scheduler for Fog Computing

5.7 Summary

In this chapter, we described the **Software Requirement Specifications (SRS)** of the proposed AI-powered Task Scheduler for Fog Computing. It detailed the project scope, operating environment, user roles, and key system components. The chapter also covered the system architecture, external interfaces, and essential nonfunctional requirements that ensure performance, reliability, and scalability. Finally, the Entity Relationship Diagram (ERD) provided a structured overview of the data relationships and interactions among major system entities, forming the foundation for implementation and database development.

Chapter 6

Modeling

This chapter presents the various modeling techniques used to represent the behavior, structure, and interactions within the **AI-powered Task Scheduler for Fog Computing** system. It focuses on understanding how different users and system components interact, and how the scheduler responds through intelligent task allocation and monitoring functionalities. Modeling provides a clear and structured way to visualize, specify, and document the design of the proposed system.

6.1 Use Case Diagram

A **Use Case Diagram** is a behavioral diagram defined by UML (Unified Modeling Language) that represents the interaction between users (actors) and the system. It provides a graphical overview of the functionalities offered by the **AI-powered Task Scheduler for Fog Computing**, illustrating how various users interact with system features to achieve their goals.

In this project, the Use Case Diagram illustrates how system administrators, IoT devices, and cloud/fog nodes interact with the scheduler to perform tasks such as submitting workloads, allocating resources, monitoring performance, and visualizing real-time analytics.

Modeling Elements in Fog Computing Use Case Diagram

Actors:

- **IoT Device:** Generates data and computational tasks to be scheduled and processed.
- **Fog Node:** Executes latency-sensitive tasks assigned by the AI scheduler.
- **Cloud Node:** Handles complex or computation-heavy tasks offloaded from fog nodes.
- **System Administrator:** Monitors overall system performance and manages configurations through the dashboard.

Use Cases:

- Generate and Submit IoT Tasks
- Perform AI-Based Task Scheduling
- Allocate Tasks to Fog or Cloud Nodes
- Monitor Resource Utilization
- Optimize Scheduling via Multi-objective Algorithm
- Visualize Performance Dashboard
- Update Scheduling Parameters
- Manage System Logs and Reports

Associations:

- Represent relationships between **actors** and **use cases**.
- For example, the **IoT Device** interacts with the *Generate and Submit IoT Tasks* use case, while the **System Administrator** interacts with *Monitor Resource Utilization* and *Visualize Performance Dashboard*.

Include Relationship:

- The "include" relationship represents dependency among use cases.
- In this project:
  - The **Perform AI-Based Scheduling** use case *includes* **Allocate Tasks to Fog or Cloud Nodes**.
  - The **Monitor Resource Utilization** use case *includes* **Visualize Performance Dashboard**.

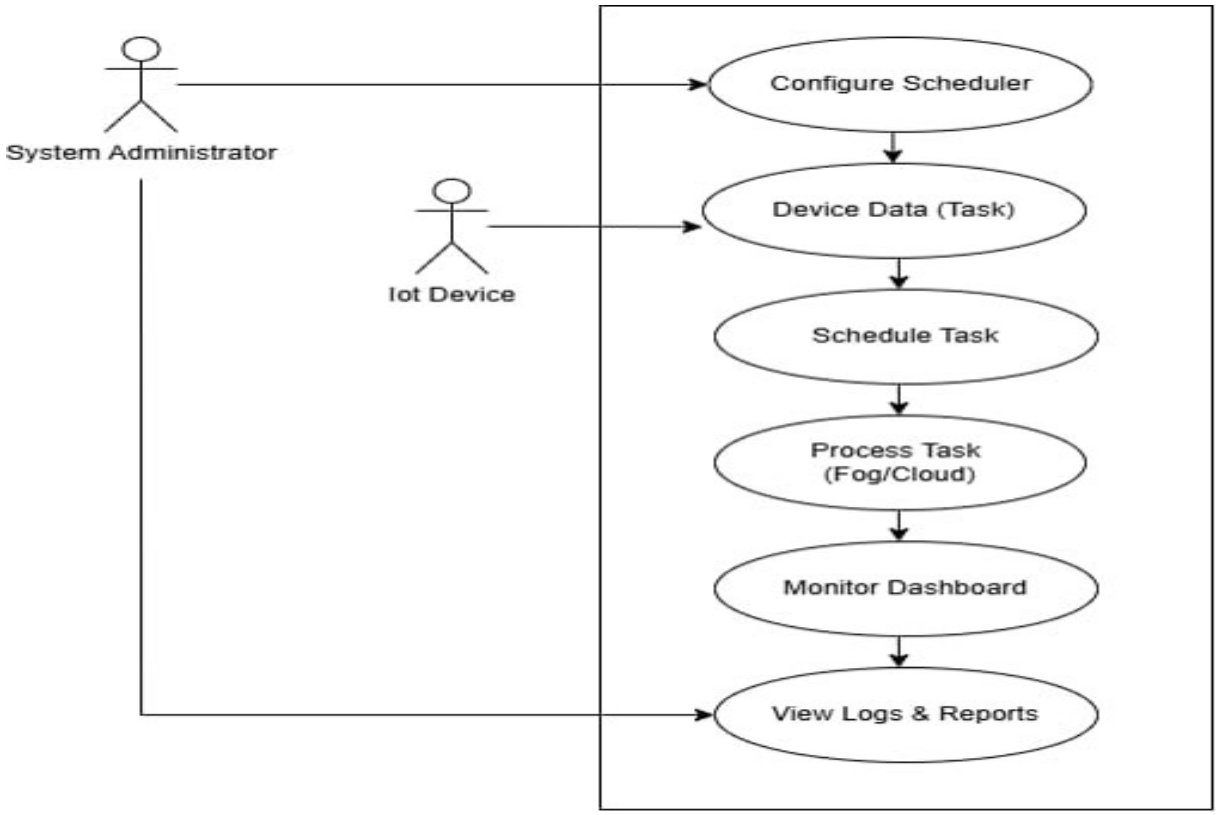


Figure 6.1: Use Case Diagram of AI-powered Task Scheduler for Fog Computing

6.2 Class Diagram

The **Class Diagram** represents the static structure of the **AI-powered Task Scheduler for Fog Computing** system by showing the system's classes, their attributes, methods, and relationships. It defines the major components, data entities, and their interactions, serving as a blueprint for system implementation.

The Class Diagram includes key classes such as *IoTDevice*, *Task*, *FogNode*, *CloudNode*, *AIScheduler*, *Optimizer*, and *PerformanceMetrics*. The diagram depicts associations among these classes—showing how the scheduler receives tasks, decides their allocation, and updates performance logs. Aggregations and dependencies indicate the data flow and relationships among entities, ensuring modularity and scalability in the system.

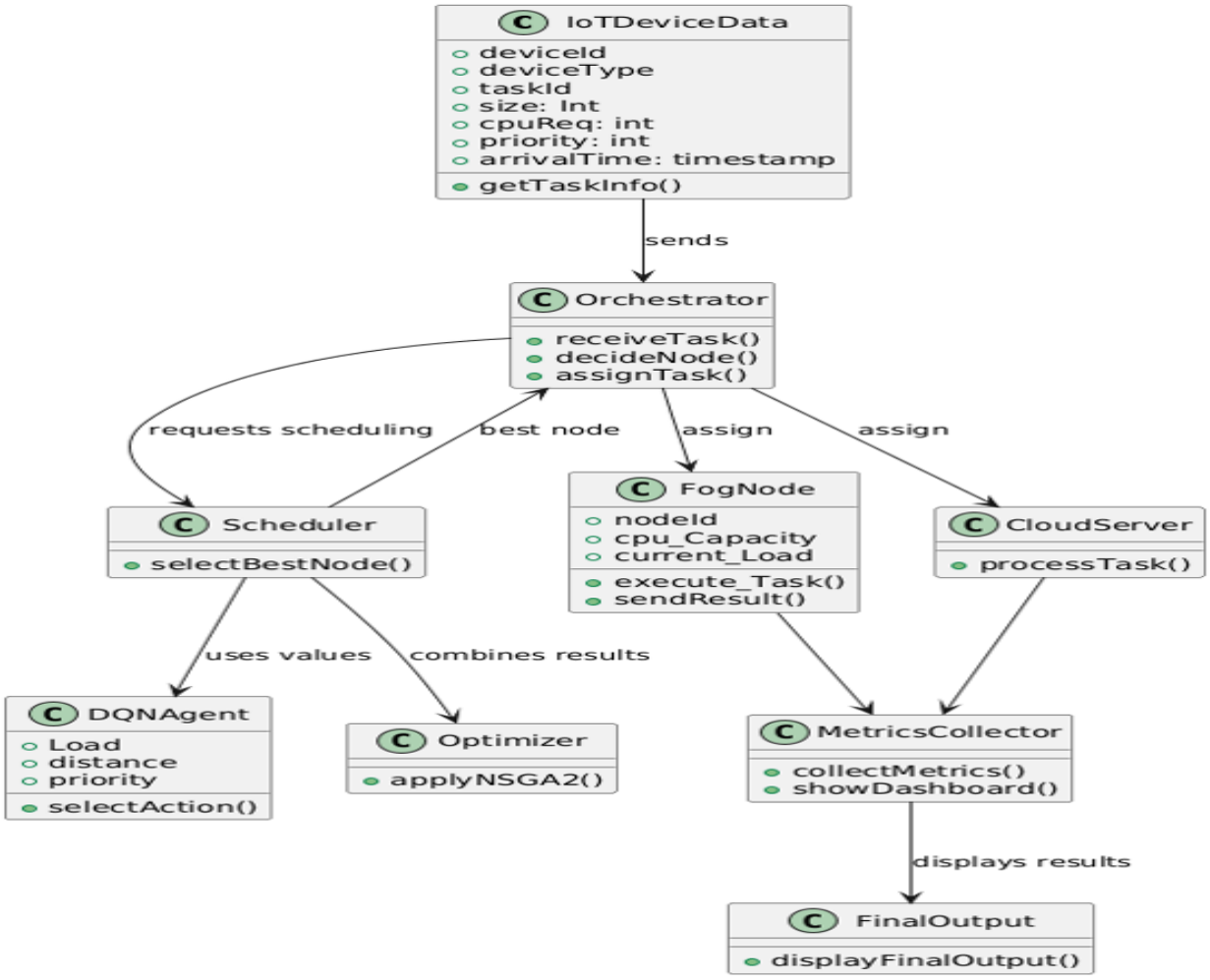


Figure 6.2: Class Diagram of AI-powered Task Scheduler for Fog Computing

6.3 Sequence Diagram

The **Sequence Diagram** depicts the chronological sequence of interactions between various components in the system. It describes how messages are passed between entities during task scheduling, allocation, and monitoring in the **AI-powered Task Scheduler for Fog Computing**.

The sequence starts when an *IoT Device* submits a task to the *AIScheduler*. The scheduler communicates with the *Optimizer* to determine the best node (fog or cloud) based on system conditions. Once the decision is made, the task is sent to the chosen node for execution. Finally, the results are sent to the *PerformanceMonitor*, which updates the dashboard and stores the performance metrics for analysis. This diagram captures both synchronous and asynchronous interactions among modules.

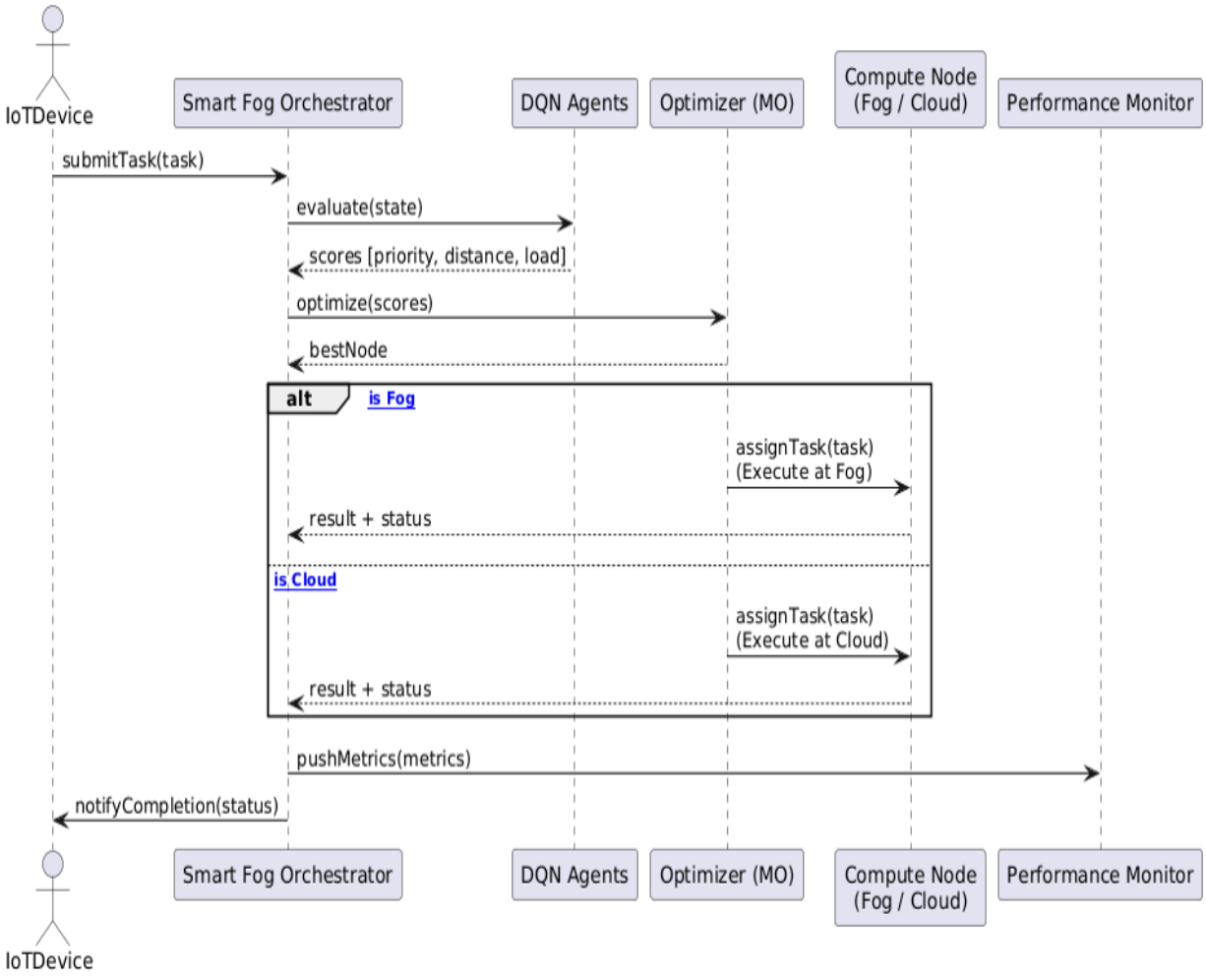


Figure 6.3: Sequence Diagram of AI-powered Task Scheduler for Fog Computing

6.4 Summary

This chapter presented the various UML modeling techniques used to design the **AI-powered Task Scheduler for Fog Computing**. The models included the Use Case, Class, Activity, and Sequence Diagrams, which collectively demonstrate the functional behavior, data structure, and interactions between system components. These diagrams provide a visual foundation for implementing the proposed intelligent scheduling system that efficiently manages IoT tasks across fog and cloud environments.

Chapter 7

Implementation and Results

The proposed system titled **"AI-powered Task Scheduler for Fog Computing"** is implemented using a modular approach. The system integrates Deep Reinforcement Learning (DQN) and Multi-Objective Optimization (NSGA-II) to efficiently schedule tasks between fog and cloud environments. The implementation and obtained results are discussed below.

7.1 Module-wise Implementation

7.1.1 IoT Dataset Module

The IoT Dataset Module is responsible for collecting and preparing task data.

Implementation Details:

- Collected IoT task parameters such as task size, CPU requirement, priority, and distance.
- Applied preprocessing, normalization, and feature extraction.

Result: The module successfully generated a structured dataset used for training and testing the scheduling model.

7.1.2 Task Environment Creation Module

This module simulates the fog and cloud infrastructure.

Implementation Details:

- Created fog nodes and cloud servers
- Simulated network latency and resource availability
- Generated dynamic task arrival scenarios

Result: A realistic simulation environment was created, enabling accurate evaluation of scheduling strategies.

7.1.3 DRL-Based Scheduling Module

This module implements Deep Q-Network (DQN) for intelligent scheduling.

Implementation Details:

- Observed system state (CPU load, latency)
- Selected actions (fog or cloud allocation)
- Used reward-based learning for optimization

Result: The DRL model learned optimal scheduling policies, resulting in improved task allocation and reduced response time.

7.1.4 Multi-Objective Optimization Module (NSGA-II)

This module optimizes multiple performance parameters.

Implementation Details:

- Applied NSGA-II for Pareto optimization
- Considered latency, CPU load, energy, and resource usage

Result: Achieved balanced and optimized scheduling solutions with improved overall system performance.

7.1.5 Performance Monitoring Module

This module provides visualization and analysis.

Implementation Details:

- Developed dashboard using React.js
- Displayed real-time metrics and graphs

Result: The system provided clear insights into performance metrics such as latency, CPU utilization, and task completion time.

7.2 Implementation Snapshots

Following are the snapshots of AI-powered Task Scheduler for Fog Computing System

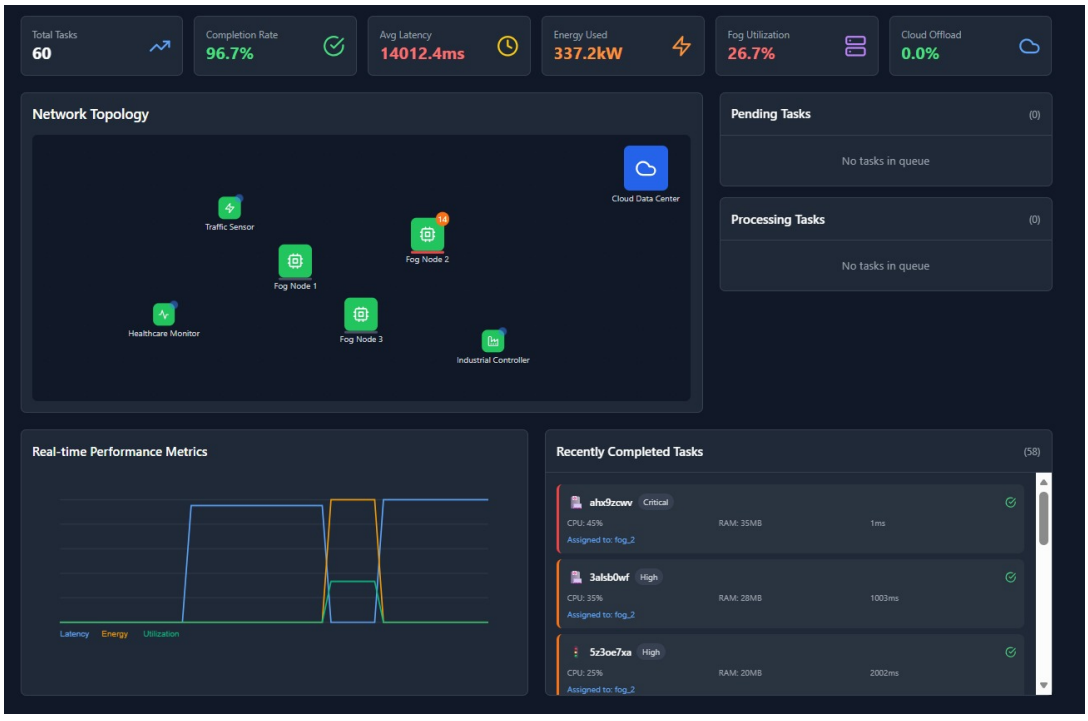


Figure 7.1: AI-powered Task Scheduler for Fog Computing System Interface

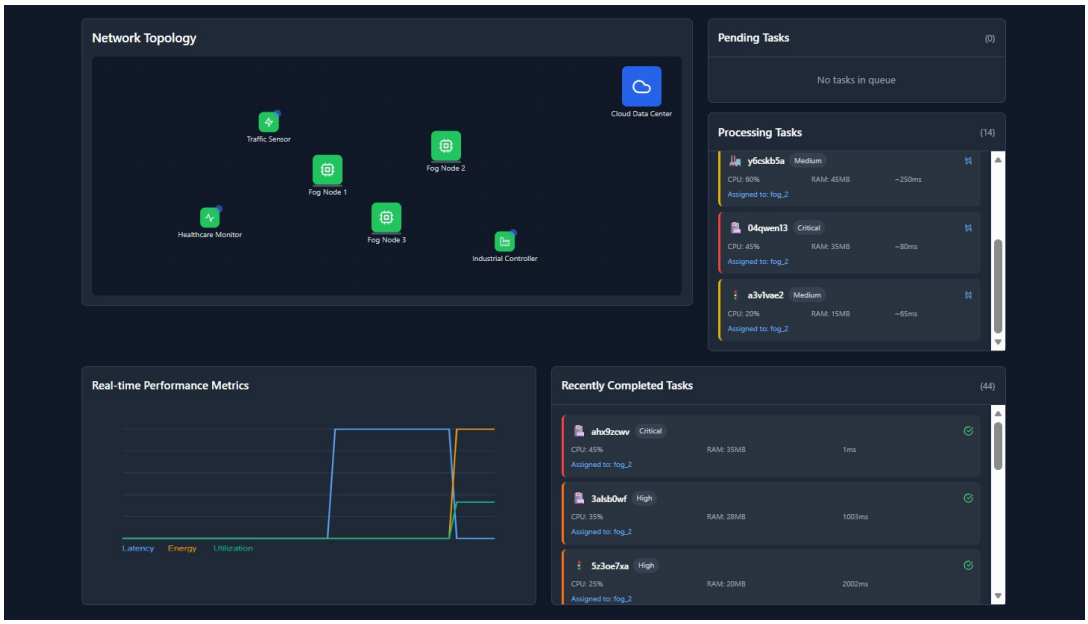


Figure 7.2: Task Environment Creation



Figure 7.3: Performance Monitoring Dashboard

7.3 Overall Results and Analysis

7.3.1 Performance Improvements

The implemented system shows the following improvements:

- Reduced latency due to fog-based processing
- Faster task completion time
- Improved CPU load balancing
- Efficient utilization of resources

7.3.2 Scalability Analysis

The system was tested by increasing the number of fog nodes:

- Initial setup: 3 fog nodes
- Extended setup: 10 and 20 fog nodes

Observations:

- Latency decreased as the number of nodes increased
- Task execution became faster
- Load distribution improved significantly

7.3.3 Optimization Results

Using NSGA-II:

- Generated Pareto-optimal solutions
- Balanced trade-offs between latency, energy, and CPU utilization

7.4 Summary

The system dynamically adapts to changing workloads and network conditions. It ensures efficient and reliable scheduling for real-time IoT applications in fog computing environments. The results demonstrate that the proposed approach is scalable, intelligent, and effective.

Chapter 8

Testing

Testing is conducted to ensure the correctness, performance, and robustness of the proposed system.

8.1 Testing Strategy

8.1.1 Unit Testing

Each module is tested independently to verify functionality.

- IoT Dataset Module
- Task Environment Module
- DRL Scheduling Module
- Optimization Module

8.1.2 Integration Testing

Ensures proper interaction between modules.

Dataset → Environment → DRL Scheduler → Optimization → Dashboard

8.1.3 Performance Testing

Evaluates system performance using key metrics:

- Latency
- Task completion time
- CPU utilization
- DRL reward efficiency

8.1.4 Edge Case Testing

The system is tested under extreme conditions:

- High task load
- Limited fog nodes
- High network latency
- Sudden burst of tasks

8.2 Test Cases

Test ID	Scenario	Input	Expected Output	Result
TC01	Basic Scheduling	Few tasks, 3 nodes	Correct assignment	Pass
TC02	High Load	Large number of tasks	No system failure	Pass
TC03	Priority Scheduling	Mixed priority tasks	High priority executed first	Pass
TC04	Limited Resources	Few fog nodes	Tasks redirected to cloud	Pass
TC05	Optimization Check	Multi-objective input	Balanced solution	Pass
TC06	Integration Testing	Full system pipeline	Smooth data flow	Pass

Table 8.1: Test Cases for System Validation

8.3 Summary

The system was tested using unit, integration, performance, and edge case testing techniques to ensure reliability and efficiency. All modules functioned correctly and showed smooth data flow across the system. Performance testing confirmed reduced latency, improved task completion time, and balanced CPU utilization. The system also handled extreme conditions like high load and limited resources effectively. All test cases passed successfully, proving the system is robust and scalable.

Chapter 9

Technical Specifications

In this chapter, we discuss the applications of the proposed framework and the detailed technical requirements essential for implementing the **AI-powered Task Scheduler for Fog Computing** system.

9.1 Applications

The **AI-powered Task Scheduler for Fog Computing** can be applied in a wide range of domains that require efficient resource allocation, latency minimization, and energy optimization. The main applications are as follows:

- **Smart Cities:** Used for managing large-scale IoT devices such as traffic sensors, surveillance systems, and smart lighting by optimizing real-time task distribution among fog nodes.
- **Healthcare Monitoring:** Enables real-time patient data processing and alert generation with minimal latency through distributed fog computing.
- **Industrial Automation:** Supports predictive maintenance, real-time analytics, and process automation by distributing tasks among local fog and cloud servers.
- **Smart Agriculture:** Helps in real-time monitoring of environmental conditions such as temperature, humidity, and soil health using AI-driven fog-based decision systems.

- **Autonomous Vehicles:** Assists in low-latency processing for vehicular networks, enabling quick decision-making for navigation, collision avoidance, and route optimization.
- **Cloud Service Optimization:** Improves load balancing between edge, fog, and cloud layers, reducing energy consumption and ensuring efficient task execution.

9.1.1 Hardware Requirements

- Processor: Intel Core i5/i7 or AMD Ryzen 5/7 (Minimum 2.4 GHz)
- RAM: 8 GB or higher (Recommended 16 GB for AI training)
- Storage: Minimum 256 GB SSD or 500 GB HDD
- GPU: NVIDIA CUDA-enabled GPU (optional for AI model optimization)
- Network: Stable internet connection for cloud-fog communication
- Edge/Fog Devices: Raspberry Pi, Jetson Nano, or equivalent embedded systems

9.1.2 Software Requirements

- Programming Languages: Python, JavaScript
- Frameworks: TensorFlow / PyTorch, Flask, Node.js
- Development Tools: VS Code, PyCharm, Jupyter Notebook, Docker
- Simulation Tools: iFogSim, CloudSim, or EdgeCloudSim
- Databases: MySQL / Firebase / MongoDB
- Operating Systems: Ubuntu / Windows 10 or higher
- Cloud Platforms: AWS, Google Cloud, or Microsoft Azure

9.2 Summary

In this chapter, we discussed the diverse applications of the **AI-powered Task Scheduler for Fog Computing** framework, highlighting its relevance across multiple real-time computing domains. We also outlined the essential hardware and software requirements necessary for system development and deployment. The proposed architecture demonstrates scalability, reliability, and adaptability in managing distributed IoT workloads.

Chapter 10

Future Scope

The project **AI-powered Task Scheduler for Fog Computing** has proven to be a vital step toward efficient IoT task management and intelligent resource utilization. Although the current system effectively handles real-time scheduling and optimization, several future improvements and extensions can enhance its performance, flexibility, and integration capabilities. The major future scopes are as follows:

- **Integration with Edge AI:** The future version can implement AI models directly on fog and edge devices, enabling local inference and reducing dependency on centralized servers.
- **Dynamic Resource Prediction:** The inclusion of reinforcement learning can allow the scheduler to predict future workload demands and allocate resources dynamically for optimal performance.
- **Blockchain-based Security:** Implementing blockchain for data integrity and secure task transmission between fog nodes and cloud servers will enhance reliability.
- **Energy-aware Optimization:** Further enhancement can include energy-aware models that balance task performance with minimal power consumption across distributed nodes.
- **Support for Multi-fog Collaboration:** Introducing inter-fog communication will allow nodes to share workload information, ensuring better distribution and load balancing.

- **Scalability toward 6G and IoT 2.0 Networks:** The scheduler can be upgraded to support ultra-low latency communication and massive IoT connectivity expected in next-generation wireless networks.
- **AI Explainability and Visualization:** Integration of Explainable AI (XAI) tools to visualize scheduling decisions and task prioritization will enhance transparency and trust.

Thus, the system can evolve into a fully autonomous, secure, and adaptive fog computing solution that supports real-time distributed intelligence across industries, healthcare, and urban infrastructure.

Chapter 11

Conclusion

In today's rapidly advancing IoT environment, efficient resource management and task scheduling are critical challenges due to the increasing demand for low-latency and high-performance applications. The proposed project **AI-powered Task Scheduler for Fog Computing** successfully addresses these challenges through intelligent decision-making and dynamic task allocation between fog and cloud layers.

The system integrates Artificial Intelligence techniques with Fog Computing to achieve optimized task distribution, latency reduction, and energy efficiency. Using AI-driven algorithms, the scheduler evaluates task characteristics such as priority, deadline, and complexity to assign them to the most suitable node. The architecture ensures scalability, fault tolerance, and adaptability across heterogeneous computing environments.

Experimental results and simulation outcomes demonstrate significant improvements in task execution time, energy utilization, and system throughput compared to traditional schedulers. Moreover, the modular design of the framework allows easy integration with cloud APIs and future AI models, ensuring sustainability in evolving technological landscapes.

Overall, the **AI-powered Task Scheduler for Fog Computing** contributes a robust and intelligent solution for real-time distributed computing, paving the way for next-generation smart systems that leverage AI, IoT, and Fog Computing synergy for efficient and sustainable performance.

Appendix A

Achievements

- **Poster Competition :** The poster competition provided a platform to showcase our project's ob jectives, methodology, and outcomes in a visually appealing and concise format. Presenting our work through a poster allowed us to communicate our ideas to a broader audience, gaining insightful feedback from peers and experts. This expe rience enhanced our presentation skills and gave us an opportunity to refine our project's focus, making it more impactful and accessible to different stakeholders.

Appendix B

Certificates

Poster Presentation Certificate:



BHUJBAL KNOWLEDGE CITY
Mumbai Educational Trust

INSTITUTE OF ENGINEERING, NASHIK
DEPARTMENT OF COMPUTER ENGINEERING

CERTIFICATE
OF PARTICIPATION

This Is To Certify That

Mr. / Ms. Atul Bhausaheb Sanap of Department of Computer Engineering has successfully presented project poster on AI-powered Task Scheduler for Fog Computing at Project Poster Presentation Competition.
An Institute Level Event held at MET's Institute of Engineering, Nashik on 16th October 2025.

SAC Coordinator
Prof. Anand Gharu

Dr. P. M. Yawalkar
HOD (Computer Engineering)

Dr. V. P. Wani
Principal



INSTITUTE OF ENGINEERING, NASHIK
DEPARTMENT OF COMPUTER ENGINEERING

BHUJBAL KNOWLEDGE CITY
Mumbai Educational Trust

SAC

CERTIFICATE
OF PARTICIPATION

This Is To Certify That

Mr. / Ms. Om Kiran Sawant of Department of Computer Engineering has successfully presented project poster on AI-powered Task Scheduler for Fog Computing at Project Poster Presentation Competition.
An Institute Level Event held at MET's Institute of Engineering, Nashik on 16th October 2025.

SAC Coordinator
Prof. Anand Gharu

Dr. P. M. Yawalkar
HOD (Computer Engineering)

Dr. V. P. Wani
Principal



INSTITUTE OF ENGINEERING, NASHIK
DEPARTMENT OF COMPUTER ENGINEERING

BHUJBAL KNOWLEDGE CITY
Mumbai Educational Trust

SAC

CERTIFICATE
OF PARTICIPATION

This Is To Certify That

Mr. / Ms. Gunjan Sanjay Shahade of Department of Computer Engineering has successfully presented project poster on AI-powered Task Scheduler for Fog Computing at Project Poster Presentation Competition.
An Institute Level Event held at MET's Institute of Engineering, Nashik on 16th October 2025.

SAC Coordinator
Prof. Anand Gharu

Dr. P. M. Yawalkar
HOD (Computer Engineering)

Dr. V. P. Wani
Principal



BHUJBAL KNOWLEDGE CITY
Mumbai Educational Trust

INSTITUTE OF ENGINEERING, NASHIK
DEPARTMENT OF COMPUTER ENGINEERING

SAC

CERTIFICATE
OF PARTICIPATION

This Is To Certify That

Mr. / Ms. Shailesh Prabhakar Pawar of Department of Computer Engineering has successfully presented project poster on AI-powered Task Scheduler for Fog Computing at Project Poster Presentation Competition.
An Institute Level Event held at MET's Institute of Engineering, Nashik on 16th October 2025.

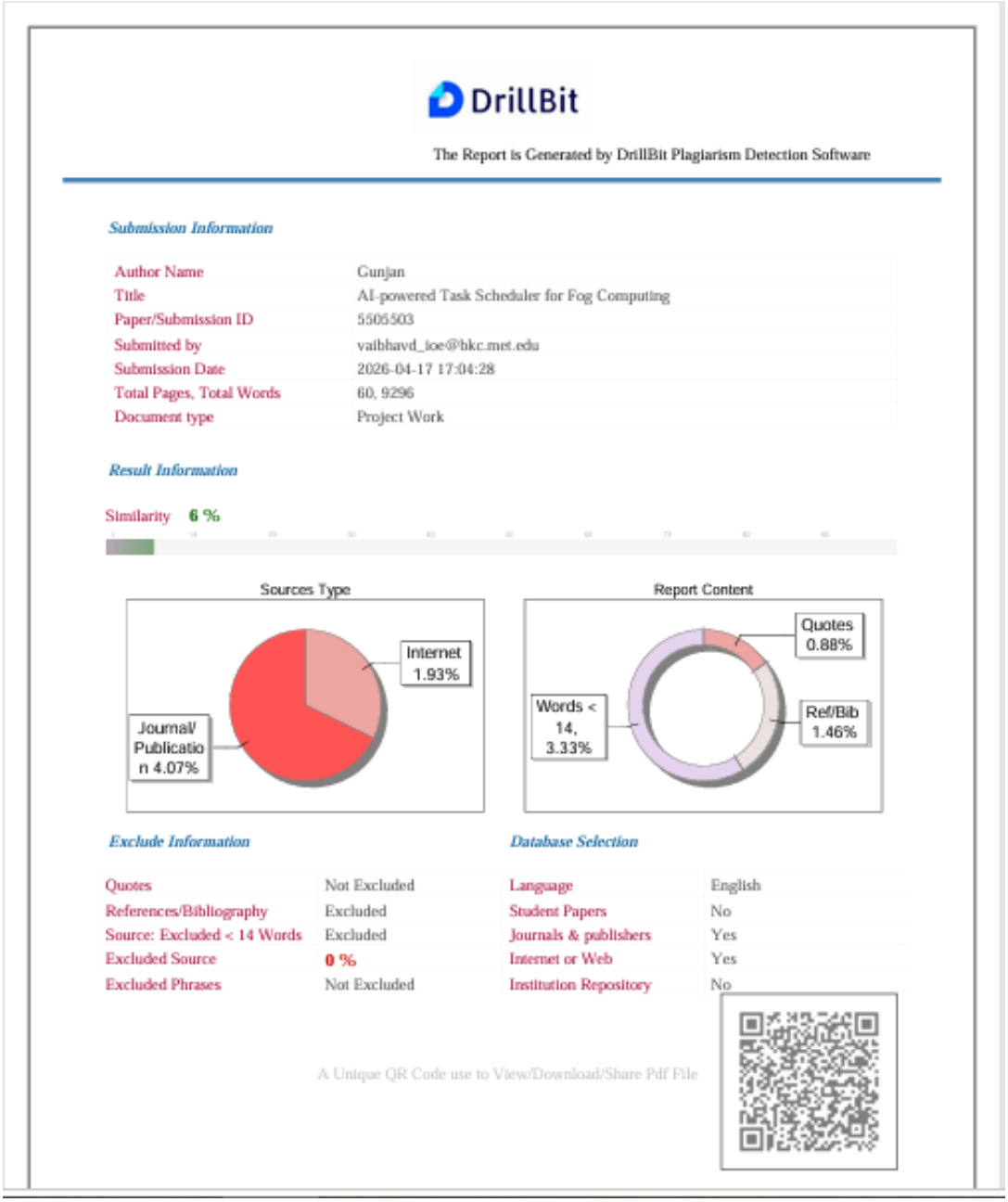
SAC Coordinator
Prof. Anand Gharu

Dr. P. M. Yawalkar
HOD (Computer Engineering)

Dr. V. P. Wani
Principal

Appendix C

Plagiarism Report



DrillBit

The Report is Generated by DrillBit Plagiarism Detection Software

Submission Information

Author Name	Gunjan
Title	AI-powered Task Scheduler for Fog Computing
Paper/Submission ID	5505503
Submitted by	vaibhavd_ioe@bkc.met.edu
Submission Date	2026-04-17 17:04:28
Total Pages, Total Words	60, 9296
Document type	Project Work

Result Information

Similarity 6 %

Sources Type

Report Content

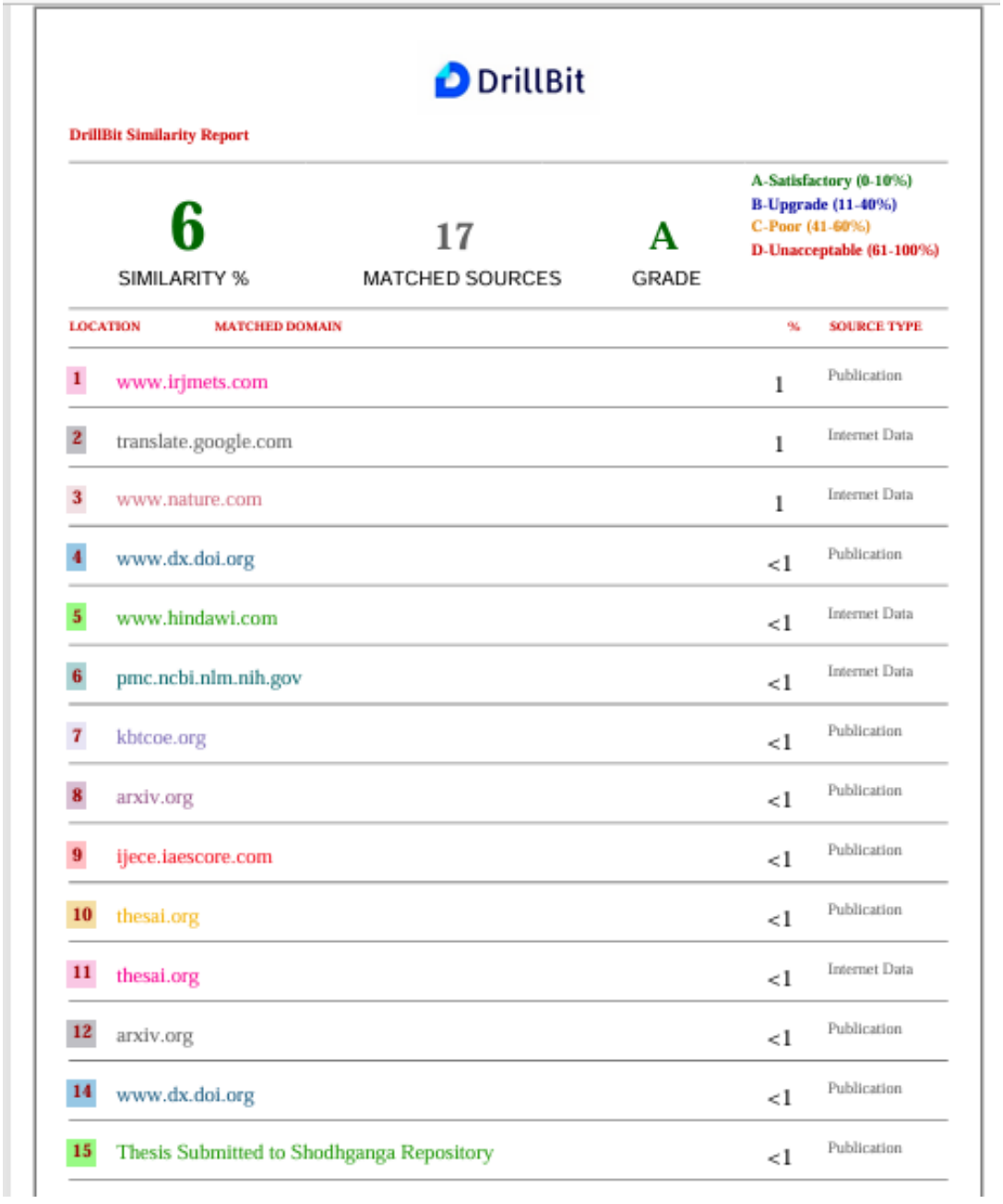
Exclude Information

Quotes	Not Excluded
References/Bibliography	Excluded
Source: Excluded < 14 Words	Excluded
Excluded Source	0 %
Excluded Phrases	Not Excluded

Database Selection

Language	English
Student Papers	No
Journals & publishers	Yes
Internet or Web	Yes
Institution Repository	No

A Unique QR Code use to View/Download/Share Pdf File



DrillBit

DrillBit Similarity Report

6	17	A
SIMILARITY %	MATCHED SOURCES	GRADE

A-Satisfactory (0-10%)
B-Upgrade (11-40%)
C-Poor (41-60%)
D-Unacceptable (61-100%)

LOCATION	MATCHED DOMAIN	%	SOURCE TYPE
1	www.irjmets.com	1	Publication
2	translate.google.com	1	Internet Data
3	www.nature.com	1	Internet Data
4	www.dx.doi.org	<1	Publication
5	www.hindawi.com	<1	Internet Data
6	pmc.ncbi.nlm.nih.gov	<1	Internet Data
7	kbtcoe.org	<1	Publication
8	arxiv.org	<1	Publication
9	ijece.iaescore.com	<1	Publication
10	thesai.org	<1	Publication
11	thesai.org	<1	Internet Data
12	arxiv.org	<1	Publication
14	www.dx.doi.org	<1	Publication
15	Thesis Submitted to Shodhganga Repository	<1	Publication

Appendix D

Team Information

Table D.1: Team Information

Sr.No.	Name of Member	Email ID	Mobile Number	Role in the Project
1				
2				
3				
4				

Bibliography

[1] Kaur, Navdeep, et al. "AI-driven Fog Computing Framework for IoT Task Scheduling and Resource Optimization." *IEEE Access*, vol. 12, 2024, pp. 10567–10579.

[2] Zhang, Lei, et al. "Machine Learning-based Task Allocation in Fog and Edge Environments." *Future Generation Computer Systems*, vol. 142, 2023, pp. 492–505.

[3] Gupta, H., Dastjerdi, A.V., et al. "iFogSim: A Toolkit for Modeling and Simulation of Resource Management Techniques in the Internet of Things, Edge and Fog Computing Environments." *Software: Practice and Experience*, vol. 47, no. 9, 2017, pp. 1275–1296.

[4] Patel, P., Shah, A., and Mehta, K. "AI-Enabled Resource Scheduling in Multi-tier Fog Architecture." *International Journal of Cloud Computing and Smart Systems*, 2024.

[5] Singh, R. and Sharma, D. "A Reinforcement Learning-based Approach for Efficient Task Scheduling in Fog Networks." *Journal of Parallel and Distributed Computing*, 2025.